



Short answer

Your design is **fundamentally correct and strong**. It demonstrates the three things the exercise is testing:

1. You understand the difference between live serving and historical research.
2. You understand point-in-time correctness and look-ahead bias.
3. You choose technologies based on workload rather than popularity.

The main improvements are to clarify the exact computation semantics of P/E, make the architecture diagram visible in the document, strengthen handling of late/corrected data, and avoid a few potentially misleading statements about “petabyte-scale” storage and cache invalidation.

The review below is based on your uploaded design document.^[1]

What the problem is asking

The central difficulty is that the inputs to a financial metric arrive at different times.

For example:

$$P/E = \frac{\text{Price}}{\text{EPS}}$$

- Price may update every few hundred milliseconds.
- EPS may change only when a quarterly result is released.
- Book value may change annually.
- Historical prices and fundamentals must be joined according to what was known at that historical moment.

There are therefore two different use cases:

Live use case

A user opens a stock page today. The system should use:

- The latest available price.
- The latest currently valid EPS.
- Current derived values such as P/E, yield, and momentum.

The user does not need to wait for a large historical query.

Historical use case

A user asks for a five-year P/E chart. For every historical date, the system must use:

- The price on that date.
- The most recent EPS that had already been publicly released by that date.

It must not use today's EPS for a price from three years ago. That would introduce **look-ahead bias**.

Your design correctly identifies this as the most important financial correctness issue.

Q1: Storage review

Your storage choices are appropriate:

Data	Proposed storage	Assessment
Tick and EOD prices	ClickHouse	Good choice for analytical time-series scans
Fundamentals	PostgreSQL	Good choice for relational, versioned data
Latest live values	Redis	Good choice for low-latency serving
Raw archive and replay	S3-compatible object storage with Parquet	Good choice for cheap long-term retention
Event distribution	Kafka or Redpanda	Good choice for replayable, scalable ingestion

Why ClickHouse works for prices

Tick data is large and mostly append-only. Queries commonly look like:

- All prices for one symbol over a time range.
- OHLC aggregation.
- Prices for many symbols on a particular day.
- Historical chart generation.

ClickHouse is suitable because it uses columnar storage, compression, partitioning, and efficient analytical scans.

Your proposed ordering is sensible:

- Tick table: (symbol, event_time).
- EOD table: (symbol, date).

You should make the partition strategy slightly more explicit. A practical design could be:

```
PARTITION BY trading_date
ORDER BY (symbol, event_time)
```

For EOD data:

```
PARTITION BY year(date)
ORDER BY (symbol, date)
```

The exact choice depends on expected query patterns, but the important principle is to avoid creating one partition per symbol or one tiny partition per day-symbol combination.

Why PostgreSQL works for fundamentals

Fundamentals are much smaller than tick data, but they require:

- Constraints.
- Transactions.
- Audit history.
- Corrections and restatements.
- Relationships to companies, instruments, exchanges, and fiscal periods.

PostgreSQL is a reasonable choice for this metadata and versioned fundamental history.

A useful refinement would be to distinguish between:

1. The **raw fundamental filing or event**, stored immutably in object storage.
2. The **normalized fundamental version**, stored in PostgreSQL.
3. The **derived TTM values**, which may be calculated and materialized separately.

This ensures that if your calculation logic changes, you can rebuild normalized or derived values from the original filing.

Point-in-time correctness

This is the strongest part of your design.

Your fields are appropriate:

```
symbol
period_end
released_at
available_at
ingested_at
version
eps_ttm
revenue
book_value
```

The most important field is `available_at`.

Suppose Q3 results are released at 7:00 PM on Tuesday:

- A calculation at 6:00 PM Tuesday must not use Q3 EPS.

- A calculation at 7:01 PM Tuesday may use it, assuming the release was actually available to the market.
- A calculation at 10:00 AM Wednesday must also use it.

The historical query should conceptually perform:

```
SELECT latest fundamental
WHERE symbol = price.symbol
      AND available_at <= price.market_timestamp
ORDER BY available_at DESC, version DESC
LIMIT 1
```

One important improvement: be precise about market timestamps and timezone handling. Store timestamps in UTC, while retaining the exchange timezone and session metadata. For NSE and BSE, you should also distinguish:

- Event publication time.
- Exchange trading timestamp.
- Ingestion time.
- The first trading session during which the information could reasonably affect trading.

This matters because a result released after market close may be considered available for the next trading session rather than for the release date's closing price.

Corrections and restatements

You correctly say that corrections should create a new version instead of overwriting history.

You should make the version semantics explicit:

- `released_at`: when the company released the information.
- `available_at`: when the platform considers it usable for investment decisions.
- `effective_from`: optional field for business applicability.
- `supersedes_version`: which earlier version was replaced.
- `is_restatement`: whether the record is a correction or restatement.

A restatement usually should not rewrite what was knowable in the past. It becomes available only from the restatement date onward for point-in-time analysis.

Archival and downsampling

Your tiered retention plan is correct:

- Recent raw ticks in ClickHouse.
- Long-term bars and EOD data queryable.
- Older raw ticks in object storage.
- Parquet files for replay and research.

Your OHLC strategy should be described more concretely. For example:

Age	Retention
0–90 days	Raw ticks in ClickHouse
90 days–2 years	Raw ticks in compressed Parquet, optional query tier
Multiple years	1-minute and 5-minute bars
Full research history	Daily OHLCV and corporate-action-adjusted prices

The phrase “petabyte-scale” may be an exaggeration depending on tick payload size, actual symbol coverage, trading hours, and retention period. However, the architectural concern is valid: retaining every tick in a hot analytical database for many years is expensive. Your tiered design addresses that correctly.

Also clarify that OHLC bars are not merely a replacement for raw data if you need:

- Market microstructure research.
- Trade-level slippage analysis.
- Order-book reconstruction.
- Tick-based volatility studies.

For ordinary charting and many factor calculations, bars are sufficient. For specialized research, raw ticks must remain available in archive storage.

Q2: Compute engine review

Your hybrid event-driven design is the right answer.

You should not choose only one of on-write, on-read, or scheduled computation. Each has a different purpose:

Computation	Recommended approach
Current price	On tick arrival
Live P/E	Event-driven, coalesced every second
Live yield	Event-driven when price or dividend changes
Historical P/E	On read, then cached/materialized
Momentum	Scheduled or incrementally updated
Backfills	Batch jobs
Corporate-action adjustments	Scheduled and versioned recomputation

Your decision to coalesce updates over approximately one second is particularly sensible. The problem explicitly says that sub-millisecond propagation is unnecessary. Computing P/E for every tick across 5,000 symbols would create unnecessary work, especially when the user-facing display only needs second-level freshness.

A typical flow is:

```
Tick arrives
→ update latest price
→ mark symbol dirty
→ wait for short debounce window
→ calculate metrics once
→ write live snapshot
```

This is better than recalculating separately for every tick.

Important improvement: avoid a global “fundamental invalidation storm”

When a new EPS arrives for one company, only that company’s live metrics should be invalidated.

For example:

```
fundamental update for RELIANCE
→ invalidate RELIANCE live metric key
→ recompute RELIANCE
```

Do not invalidate every stock or every historical chart. Use symbol-level and dataset-revision-level invalidation.

Live P/E definition

You should explicitly define which EPS is used for live P/E:

```
live P/E = latest price / latest currently valid EPS
```

But there are multiple possible EPS definitions:

- Reported annual EPS.
- TTM EPS.
- Adjusted EPS.
- Diluted EPS.
- Consolidated EPS.
- Standalone EPS.

The architecture should support a metric definition such as:

```
pe_ttm_adjusted_consolidated
```

This prevents ambiguity when users compare results.

You should also define behavior for zero or negative EPS:

- If EPS is zero: P/E is undefined.
- If EPS is negative: show “NM” or “Not Meaningful,” rather than a misleading negative P/E.
- If EPS is missing: return null with a reason code.

Q3: API and serving review

Your two API patterns are correct.

Live API

Redis is appropriate because the response should be a compact, precomputed snapshot:

```
{
  "symbol": "RELIANCE",
  "price": 2850.40,
  "eps_ttm": 112.50,
  "pe": 25.34,
  "momentum": 0.08,
  "yield": 0.42,
  "computed_at": "2026-10-01T07:02:14Z",
  "fundamental_available_at": "2026-09-30T13:30:00Z"
}
```

A Redis read should comfortably support the API latency requirement if the service, Redis cluster, and API are deployed in the same region or network.

Your fallback should be carefully controlled. If Redis misses, do not allow the API to perform an expensive historical query without a timeout. Better options are:

1. Read the latest materialized snapshot from a secondary store.
2. Return a slightly stale snapshot with a freshness timestamp.
3. Enqueue recomputation and return an explicit temporary state.

The API should expose freshness metadata so the frontend knows whether the result is current.

WebSocket or Server-Sent Events

Using REST for initial page load and WebSocket or SSE for live updates is a good pattern:

```
GET /v1/stocks/RELIANCE/metrics
→ initial snapshot

SSE /v1/stocks/RELIANCE/stream
→ subsequent updates
```

For many financial dashboards, SSE is simpler than WebSocket if communication is primarily server-to-client.

Historical API

Your historical cache-key design is good:

```
pe:v3:RELIANCE:2022-10-01:2025-10-01:daily
```

The important idea is to include a data or metric revision. Without that, a corrected fundamental could leave stale historical charts in cache.

One refinement: do not rely only on deleting old cache keys. Deletion can fail, race, or become expensive. Revisioned keys are safer:

```
historical_revision:RELIANCE = 42  
cache key = pe:v3:RELIANCE:42:2022-10-01:2025-10-01:daily
```

When relevant data changes, increment the revision. Old data naturally expires.

For a three-year chart, 750 daily points is not large. The response should be generated from an analytical query service and cached. You may optionally return fewer display points for small mobile charts, but do not destroy the underlying daily resolution used for research.

Q4: Architecture diagram

Your document says the diagram is in docs/architecture.mmd, but the uploaded text does not show the diagram itself. This is the biggest presentation weakness. The prompt explicitly asks for a high-level architecture diagram, so you should include it directly in the submission.

A suitable diagram could be:

```
flowchart LR
    B[Broker WebSocket] --> N[Normalizer and Validator]
    N --> K[Kafka / Redpanda]

    K --> T[Tick Consumer]
    K --> F[Fundamental Consumer]
    K --> E[EOD Consumer]

    T --> CH[ClickHouse Tick and EOD Tables]
    E --> CH
    F --> PG[PostgreSQL Versioned Fundamentals]

    T --> L[Latest Price State]
    L --> R[Redis Live Snapshot]
    PG --> R

    R --> LA[Live API]
    LA --> UI[Live UI]

    CH --> H[Historical Query Service]
    PG --> H
    H --> HC[Historical Cache]
```

```
HC --> HA[Historical API]
HA --> UI2[Five-Year Chart]

K --> S3[S3 / Parquet Archive]
S3 --> RP[Replay and Backfill Jobs]
RP --> CH
RP --> PG
```

You should label the two paths clearly:

Live path

```
WebSocket tick
→ Kafka
→ tick consumer
→ latest price state
→ coalesced metric calculation
→ Redis
→ live API
→ UI
```

Historical path

```
EOD price
→ ClickHouse
+
versioned fundamentals in PostgreSQL
→ as-of temporal join
→ historical cache
→ chart API
→ five-year chart
```

This makes the distinction between live serving and historical correctness immediately understandable.

Final assessment

I would rate the current design around 8/10 for the exercise.

What is already strong

- Correct separation of hot serving, analytical storage, relational fundamentals, and archive storage.
- Correct use of `available_at` for point-in-time joins.
- Correct recognition of look-ahead bias.
- Sensible second-level coalescing for live metrics.
- Good use of Redis for the under-200 ms live API.
- Good use of ClickHouse for historical price scans.

- Good retention and archival strategy.
- Good operational awareness around replay, dead-letter queues, reconciliation, and consumer lag.

What to improve before submitting

- Include the architecture diagram directly in the document.
- Define live and historical P/E precisely.
- Explain how after-market releases affect the first eligible market timestamp.
- State the exact raw-tick, bar, and EOD retention tiers.
- Explain handling of missing, zero, and negative EPS.
- Clarify that invalidation is symbol-specific.
- Distinguish raw filings, normalized fundamentals, and derived TTM values.
- Mention idempotency keys and deduplication explicitly for broker events.
- Explain how ClickHouse and PostgreSQL historical joins are executed efficiently rather than implying an unrestricted cross-database join.

With those changes, the document would not merely be technically correct; it would also be easier for an interviewer to follow and would demonstrate strong practical judgment.

✱

1. multi_frequency_financial_data_design.md